PHIẾU THỰC HÀNH — 09/10/2026

Họ tên: Huỳnh Nguyễn Tiến Anh MSSV: N24DCDT001 Lớp: D24CQDT01-N

GitHub: https://github.com/anhtienhao66a-dotcom/thuchanhbuoi5.git Commit SHA: (Xem commit HEAD trên GitHub)

1. Mô tả một luồng ứng dụng

Ghi mục tiêu, đầu vào, đầu ra và sơ đồ truyền dữ liệu giữa các chức năng.

- Mục tiêu: Điều phối giao đơn trong khuôn viên trường theo ngân sách thời gian, tối đa hóa tổng lợi ích nhận được.
- Đầu vào: 2 tệp CSV gồm don_hang.csv (id, diem_giao, thoi_gian_phuc_vu, loi_ich) và ban_do.csv (u, v - đồ thị vô hướng không trọng số).
- Đầu ra: ket_qua.csv ghi nhận chi phí, số cạnh, lợi ích, tuyến đường, trạng thái được chọn và đã giao của từng đơn hàng.
- Sơ đồ luồng dữ liệu:

CSV (don_hang, ban_do) -> read_data -> merge_sort theo ID -> Khởi tạo balanced_bst & binary_search

-> bfs_routes tính đường đi ngắn nhất từ kho (đỉnh 1) -> Xác định chi phí thời gian tau = 2h + service

-> knapsack (quy hoạch động ba lô 0/1) chọn tập đơn tối ưu trong ngân sách (đối chiếu brute_force)

-> priority_orders sắp xếp thứ tự ưu tiên (lợi ích/chi phí) -> linked_list -> CircularQueue (FIFO)

-> Quản lý giao hàng (deliver!) và hoàn tác (undo!) với ngăn xếp completed -> check_invariants -> Xuất ket_qua.csv.

2. Bảng truy vết đường và thời gian

ID	Đường lượt đi	Số cạnh h	2h + phục vụ
1	1 -> 2	1	2*1 + 1 = 3
2	1 -> 2 -> 4	2	2*2 + 1 = 5
3	1 -> 2 -> 4 -> 6	3	2*3 + 1 = 7

3. Minh chứng bốn hàm

NV1 · Hoàn thiện merge_sort(a, before)

Ý tưởng/điều kiện áp dụng: Giải thuật chia để trị (Divide and Conquer). Nếu mảng có <= 1 phần tử thì dừng và trả về bản sao. Ngược lại chia đôi mảng tại mid = div(length(a), 2), đệ quy sắp xếp hai nửa rồi trộn (merge) theo comparator before. Đảm bảo ổn định (stable): khi hai phần tử hòa nhau thì ưu tiên lấy phần tử thuộc nhánh trái trước. Không thay đổi mảng đầu vào.

Lời gọi và kết quả kiểm thử: julia TH_2026_10_09_Kiem_thu.jl --phase=sort -> ĐẠT 8/8 phép kiểm tra.

Chi phí thời gian/bộ nhớ: Thời gian O(n log n) (log2 n tầng chia, mỗi tầng trộn O(n)); bộ nhớ phụ O(n) cho mảng tạm và O(log n) ngăn xếp đệ quy.

NV2 · Hoàn thiện binary_search(a::Vector{Order}, id::Int)

Ý tưởng/điều kiện áp dụng: Tìm kiếm nhị phân trên Vector{Order} đã sắp xếp tăng dần theo ID. Duy trì đoạn tìm kiếm [lo, hi]. Lấy mid = lo + div(hi-lo, 2). Nếu a[mid].id == id trả về Order; nếu a[mid].id < id thu hẹp sang phải (lo = mid + 1); ngược lại thu hẹp sang trái (hi = mid - 1). Hết khoảng (lo > hi) trả về nothing.

Lời gọi và kết quả kiểm thử: julia TH_2026_10_09_Kiem_thu.jl --phase=search -> ĐẠT 5/5 phép kiểm tra.

Chi phí thời gian/bộ nhớ: Thời gian O(log n) do không gian tìm kiếm giảm một nửa sau mỗi bước; bộ nhớ phụ O(1) (hiện thực vòng lặp).

NV3 · Hoàn thiện bfs_routes(graph, source=1)

Ý tưởng/điều kiện áp dụng: Duyệt đồ thị theo chiều rộng (BFS) trên đồ thị vô hướng không trọng số từ nguồn source=1. Dùng CircularQueue FIFO. Khởi tạo distance=-1, parent=0. Gán distance[source]=0, parent[source]=0 và enqueue nguồn. Mỗi bước dequeue đỉnh u, duyệt đỉnh kề v: nếu distance[v]==-1 thì gán distance[v]=distance[u]+1, parent[v]=u và enqueue v ngay để tránh trùng. Sau đó truy vết ngược từ từng đỉnh về nguồn theo parent rồi đảo ngược để dựng routes[v]. Đỉnh không tới giữ distance=-1 và tuyến rỗng.

Lời gọi và kết quả kiểm thử: julia TH_2026_10_09_Kiem_thu.jl --phase=bfs -> ĐẠT 7/7 phép kiểm tra.

Chi phí thời gian/bộ nhớ: Thời gian O(V + E); bộ nhớ phụ O(V) cho distance, parent, queue (và O(V^2) cho lưu trữ các routes).

NV4 · Hoàn thiện knapsack(orders, costs, budget)

Ý tưởng/điều kiện áp dụng: Quy hoạch động giải bài toán ba lô 0/1. Bảng table[i+1, b+1] lưu lợi ích tối ưu của i đơn đầu trong ngân sách b. Công thức: nếu c_i <= b thì table[i+1, b+1] = max(table[i, b+1], v_i + table[i, b-c_i+1]); ngược lại table[i+1, b+1] = table[i, b+1]. Khi bằng nhau, ưu tiên giữ phương án không lấy đơn i. Nhánh lấy dùng hàng trước (i) để mỗi đơn chỉ lấy tối đa 1 lần. Truy vết từ table[n+1, budget+1] về để lấy ids.

Lời gọi và kết quả kiểm thử: julia TH_2026_10_09_Kiem_thu.jl --phase=dp -> ĐẠT 12/12 phép kiểm tra.

Chi phí thời gian/bộ nhớ: Thời gian O(n*B) điền bảng và O(n) truy vết; bộ nhớ phụ O(n*B) lưu bảng table.

4. Quyết định chọn đơn

Điền ngân sách, tập chọn, tổng thời gian/lợi ích. Giải thích một ô DP và một bước truy vết; đối chiếu với vét cạn.

- Bộ MINI001: Ngân sách B = 8; Tập chọn = {1, 2}; Tổng thời gian cost = 8 <= 8; Tổng lợi ích value = 14.
  + Giải thích ô F(2, 8): Xét 2 đơn {1, 2} với ngân sách 8. Không lấy đơn 2 được F(1, 8) = 6. Lấy đơn 2 (chi phí 5, lợi ích 8) được 8 + F(1, 3) = 8 + 6 = 14. Do 14 > 6 nên F(2, 8) = 14, lưu tại table[2+1, 8+1] = table[3, 9] trong Julia.
  + Giải thích truy vết: Tại i=3, b=8 có table[4, 9] = 14 == table[3, 9] = 14 -> không lấy đơn 3. Tại i=2, b=8 có table[3, 9] = 14 > table[2, 9] = 6 -> lấy đơn 2, b giảm còn 8-5=3. Tại i=1, b=3 có table[2, 4] = 6 > table[1, 4] = 0 -> lấy đơn 1, b giảm còn 3-3=0. Kết quả chọn {1, 2}.
  + Đối chiếu vét cạn: Xét 2^3 = 8 tập con, tập {1, 2} có tổng chi phí 8, lợi ích 14 là tối ưu nhất; kết quả DP hoàn toàn trùng khớp.
- Bộ mở rộng cá nhân MSSV N24DCDT001: n = 8 đơn, ngân sách B = 28. Tập chọn: [2, 8, 5, 4, 3, 7], tổng chi phí = 26 <= 28, tổng lợi ích = 45. DP = 45, Vét cạn = 45, Tham lam = 45.

5. Giao và hoàn tác

Trạng thái	Queue	Stack	Đã dùng	Đã nhận
Sau lập kế hoạch	[1, 2]	[]	0	0
Sau giao một đơn	[2]	[1]	3	6

Trạng thái	Queue	Stack	Đã dùng	Đã nhận
Sau hoàn tác	[1, 2]	[]	0	0

6. Đầu ra và nhận xét

Đường dẫn CSV kết quả: data/MINI001/ket_qua.csv và data/N24DCDT001/ket_qua.csv Link notebook: TH_2026_10_09_Pluto.jl

Lỗi đã phát hiện và cách sửa:
1. Tính ổn định của merge_sort: Khi các phần tử hòa comparator, nếu chọn nhánh phải sẽ làm mất tính ổn định. Đã sửa: Chỉ lấy nhánh phải khi before(right[j], left[i]) == true, còn lại ưu tiên lấy nhánh trái.
2. Chỉ số mảng 1-based trong Julia cho bảng knapsack: Trạng thái F(i, b) phải lưu ở table[i+1, b+1] và nhánh lấy phải tra từ hàng trước table[i, b-c+1]. Đã sửa chuẩn hóa kích thước (n+1)x(B+1) với hàng 1, cột 1 là cơ sở biên 0.

Một chức năng thay đổi ảnh hưởng đến chức năng sau như thế nào?
Hệ thống là một pipeline liên kết chặt chẽ: Thay đổi bản đồ -> bfs_routes tính lại khoảng cách -> chi phí thời gian tau = 2h + service đổi -> knapsack nhận chi phí mới và chọn tập đơn khác -> priority_orders tạo thứ tự khác -> hàng đợi FIFO pending và tiến trình deliver!/undo! thay đổi -> xuất ket_qua.csv phản ánh đúng toàn bộ trạng thái mới.

Câu hỏi cuối buổi

Q1. Vì sao không được dùng dãy đã sắp giảm theo lợi ích/τ để tìm kiếm nhị phân theo ID? Nêu tiền điều kiện của hàm tìm kiếm.

- Tiền điều kiện của tìm kiếm nhị phân: Mảng dữ liệu phải được sắp xếp theo đúng khóa tìm kiếm (ở đây là sắp xếp tăng dần theo ID đơn hàng).
- Dãy sắp giảm theo lợi ích/tau được sắp xếp theo một tiêu chí hoàn toàn khác, các ID trong dãy này không còn thứ tự tăng dần. Khi đó, phép chia đôi và so sánh nhị phân với phần tử ở giữa sẽ loại bỏ sai nhánh tìm kiếm, dẫn đến kết luận sai hoặc không tìm thấy đơn hàng.

Q2. Với đơn ID 2 tới điểm 4, phục vụ 1 và bản đồ mẫu, giải thích từng thành phần của thời gian thực hiện τ. BFS có tiếp tục cho thời gian ngắn nhất nếu các cạnh mang thời gian khác nhau không?

- Giải thích thành phần của tau: Điểm 1 là kho. Đường đi ngắn nhất từ kho 1 đến điểm 4 là 1 -> 2 -> 4 gồm h = 2 cạnh. Mỗi đơn là chuyến đi rồi về kho nên thời gian di chuyển cả hai chiều là 2*h = 2*2 = 4 đơn vị thời gian. Thời gian phục vụ tại

điểm giao là service = 1. Tổng thời gian thực hiện: tau = 2h + service = 2*2 + 1 = 5.
- BFS không tiếp tục cho thời gian ngắn nhất nếu các cạnh mang trọng số thời gian khác nhau, vì BFS giả định mọi cạnh có cùng độ dài (1 đơn vị). Với đồ thị có trọng số cạnh dương tổng quát, phải dùng thuật toán Dijkstra.

Q3. Giải thích hai phương án tạo ô F(2,8) và vị trí của ô đó trong table Julia. Vì sao nhánh lấy đơn phải dùng hàng trước?

- Hai phương án tạo F(2, 8): Phương án không lấy đơn 2 có giá trị F(1, 8) = 6. Phương án lấy đơn 2 (chi phí 5, lợi ích 8) có giá trị 8 + F(1, 8-5) = 8 + F(1, 3) = 8 + 6 = 14. Chọn max(6, 14) = 14.
- Vị trí trong Julia: Do chỉ số bắt đầu từ 1, ô F(2, 8) nằm tại table[2+1, 8+1] = table[3, 9].
- Nhánh lấy đơn phải dùng hàng trước (i-1) để đảm bảo bài toán ba lô 0/1 (mỗi đơn chỉ được chọn tối đa một lần). Nếu lấy từ cùng hàng i, một đơn có thể bị chọn lặp lại nhiều lần (ba lô không giới hạn).

Q4. Sau khi giao ID 1 rồi hoàn tác ngay, FIFO, ngăn xếp, tổng thời gian và tổng lợi ích phải thay đổi thế nào? Nêu một bất biến kiểm tra việc không mất hoặc lặp đơn.

- Trạng thái thay đổi: Khi giao đơn 1, ID 1 rời FIFO pending sang đỉnh ngăn xếp completed, spent tăng lên 3, earned tăng lên 6. Khi hoàn tác ngay, ID 1 được lấy từ đỉnh ngăn xếp và đưa ngược về đầu FIFO bằng enqueue_front!, spent giảm về 0, earned giảm về 0. FIFO trở lại [1, 2], ngăn xếp rỗng [], hệ thống khôi phục nguyên vẹn trạng thái trước khi giao.
- Bất biến kiểm tra: Set(vcat(queue_values(s.pending), s.completed)) == Set(s.selected_ids) và length(vcat(queue_values(s.pending), s.completed)) == length(s.selected_ids) (không mất, không lặp đơn).

Q5. Cho biết độ phức tạp theo n,V,E,B của sắp xếp trộn, tìm kiếm nhị phân, BFS cốt lõi và DP bảng đầy đủ. Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n)?

- Độ phức tạp: Sắp xếp trộn: thời gian O(n log n), bộ nhớ O(n). Tìm kiếm nhị phân: thời gian O(log n), bộ nhớ O(1). BFS cốt lõi: thời gian O(V + E), bộ nhớ O(V) (lưu toàn bộ routes tốn O(V^2)). DP ba lô: thời gian O(n*B), bộ nhớ O(n*B).
- Không thể khẳng định mọi BST đều tra cứu trong O(log n) vì chiều cao h của BST phụ thuộc vào thứ tự chèn phần tử. Nếu các phần tử chèn theo thứ tự tăng/giảm dần, BST bị thoái hóa thành danh sách liên kết với chiều cao h = n, khi

đó chi phí tra cứu trong trường hợp xấu nhất là O(n). BST chỉ đạt O(log n) khi cây cân bằng.